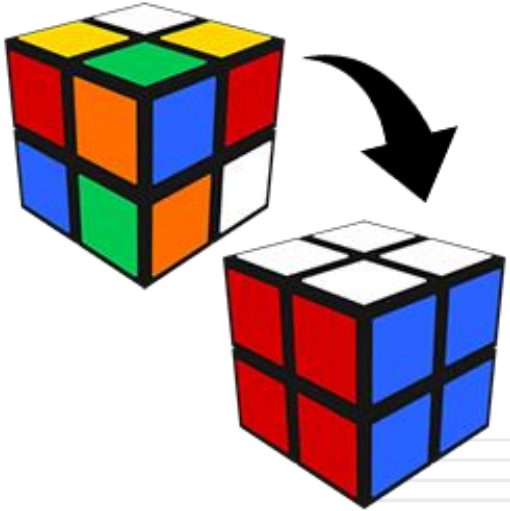
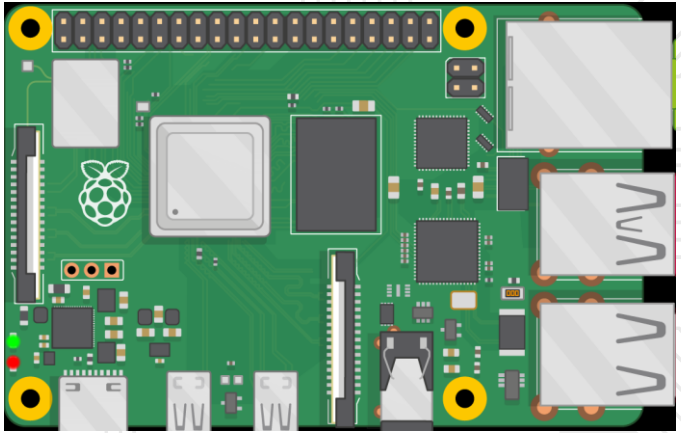
분산 처리를 이용한 문제 해결

2x2 Rubik's Cube solving system
2x2 루빅스 큐브 솔빙 시스템





팀 소개 및 역할분담

주상우 소프트웨어학과

- #1번 파이 담당
- #3번 파이 공동 담당
- 초음파 센서 2개 및 조도 센서
- LED 3개
- 발표 및 PPT

이재혁 소프트웨어학과

- #0번 파이(서버) 담당
- 버튼 5개
- 큐브 솔빙 알고리즘 구현

임상인 소프트웨어학과

- #2번 파이 담당
- 카메라
- 서보 모터 3개
- 픽셀 색상 판별 알고리즘 구현

공동 구현

- 분산 처리 시스템 구현
- 서버와 클라이언트 간 소켓 통신 구현

목 차

1 프로젝트 개요

2 하드웨어 배치 및 구조

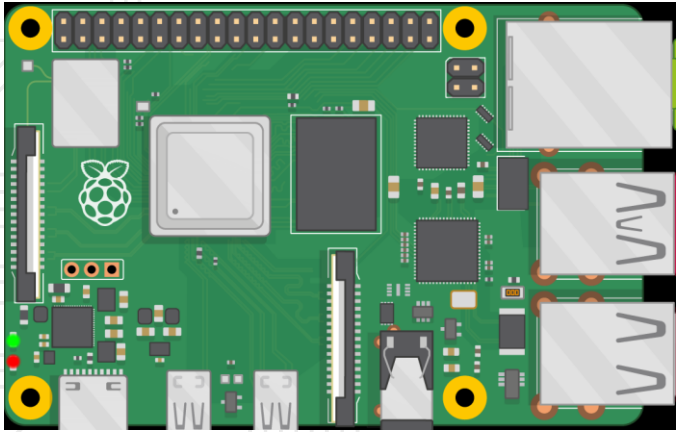
3 소프트웨어 구조 및 설계

4 기존 계획 대비 변경점

5 트러블 슈팅 및 시스템 개선 과정

6 시스템 확장 가능성

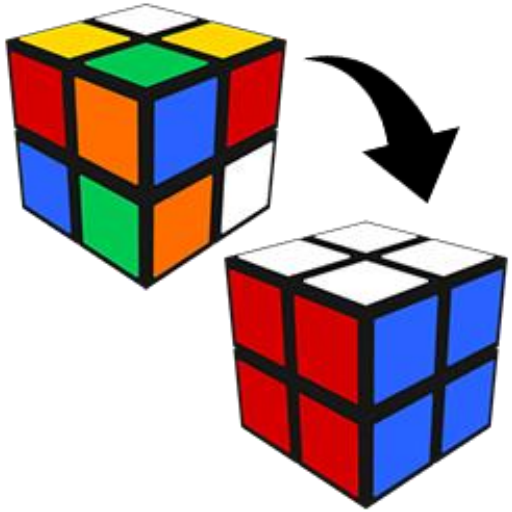
7 시연 영상



1

프로젝트 개요

프로젝트 개요



1단계: 입력

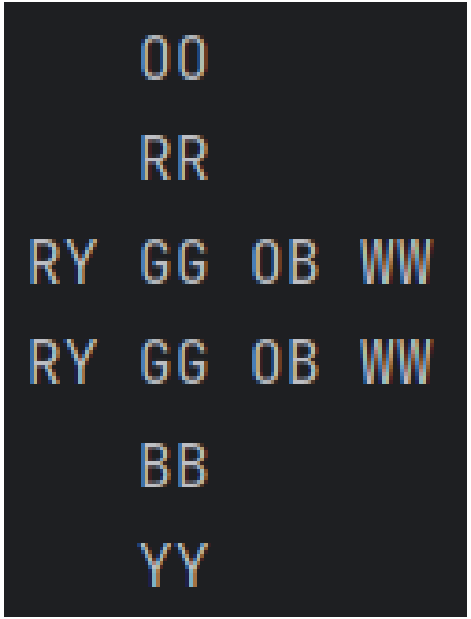
2단계: 분산 처리

3단계: 출력

1단계: 입력

카메라

섞인 큐브

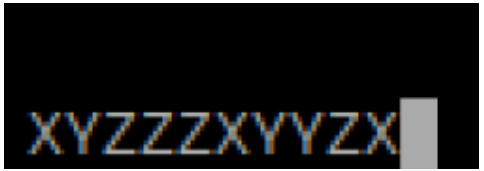


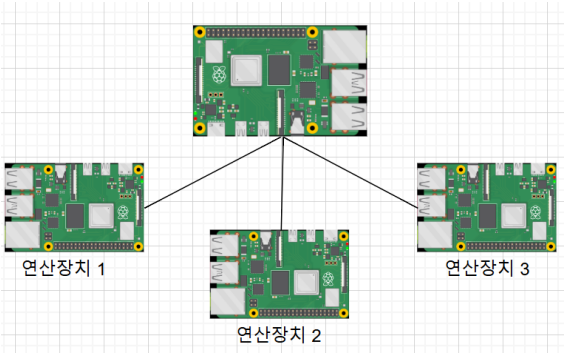
큐브 전개도

2단계: 분산 처리

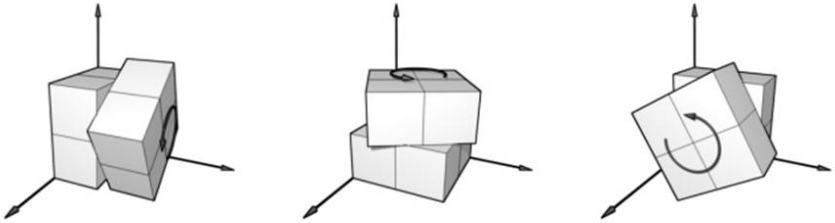
섞인 큐브

어떻게 돌려야 할지 그 솔루션을 계산.





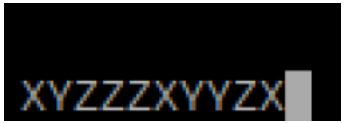
ex. 현재 큐브 상태에 대해선,
XYZZZXYYZX 순서대로 돌려야 한다.



X-axis turn *Y-axis turn* *Z-axis turn*

3단계: 출력

섞인 큐브



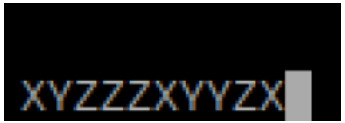
1. “터미널에 나온 대로, 큐브를 XYZZZXYYZX순서대로 돌려야 한다.”

사용자

섞인 큐브

3단계: 출력

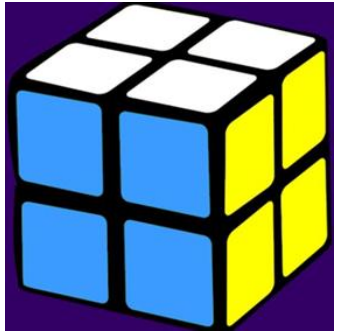
섞인 큐브



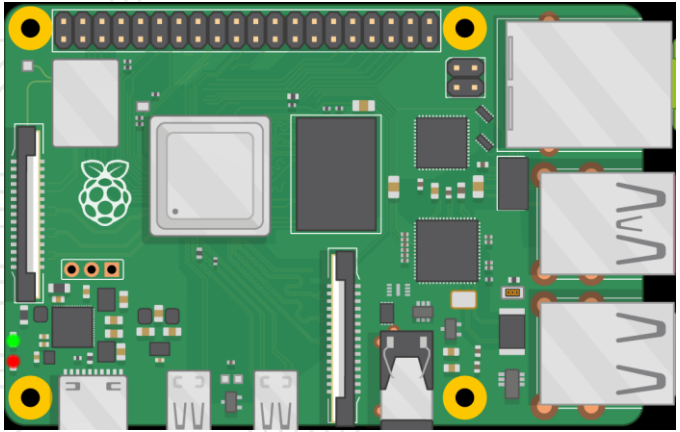
2. 결과 솔루션에 따라, 3개의 서보 모터가 직접 큐브를 회전시켜 맞춰줌.

사용자

섞인 큐브



맞춰진 큐브

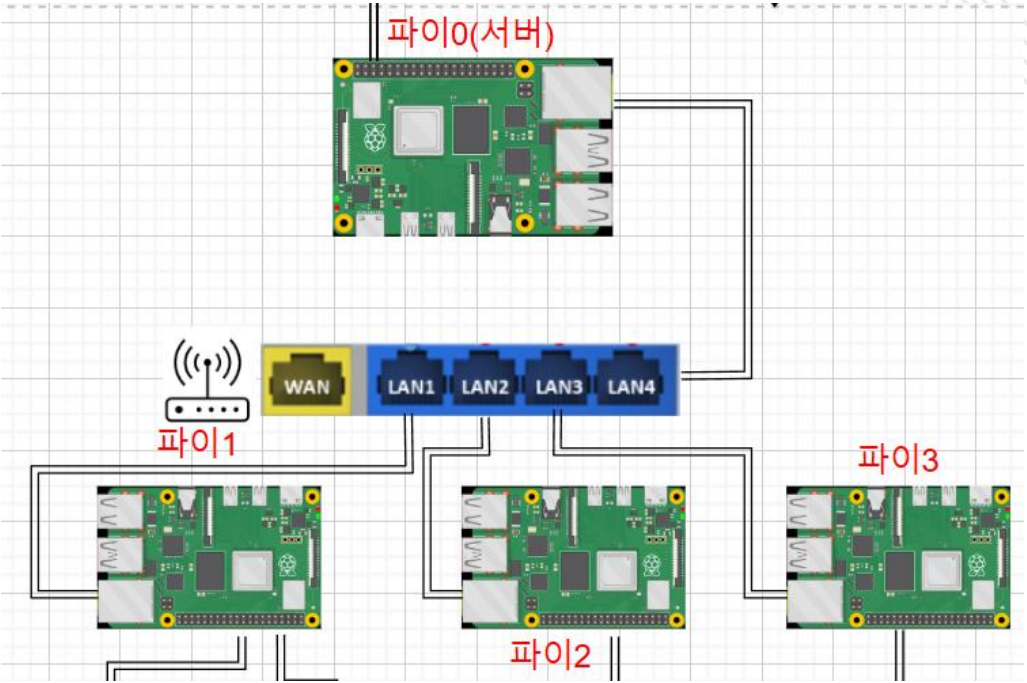


2

하드웨어 배치 및 구조

1단계: 입력

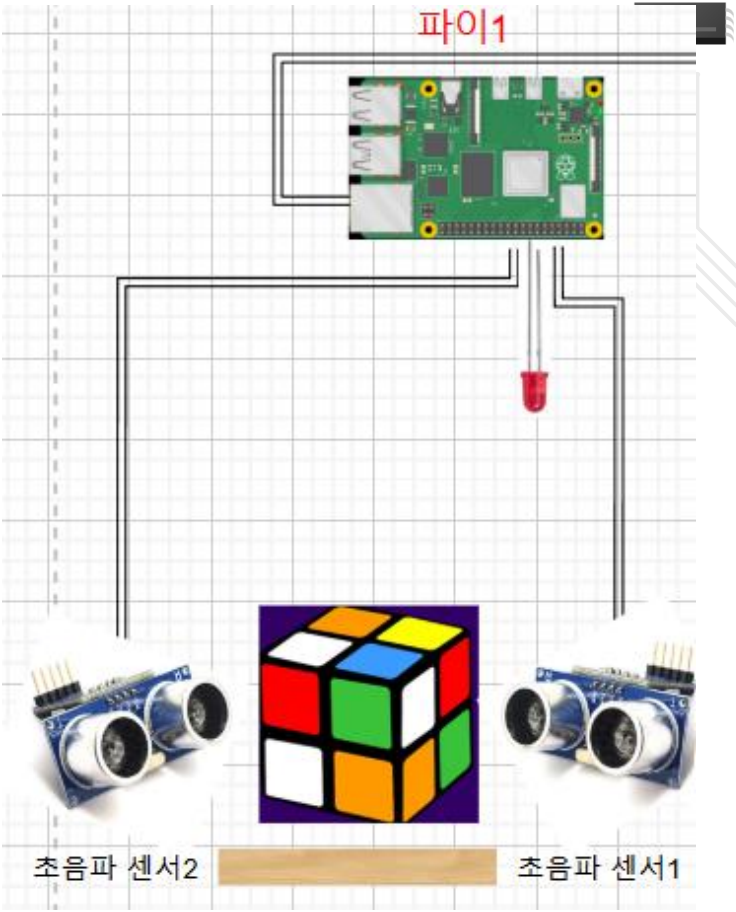
#0번 파이: 서버
#1, #2, #3번 파이: 클라이언트



#1번 파이

초음파 센서 2개로, 큐브 사진을
제대로 찍을 수 있는 거리인지 센싱

두 센서 다 적정 거리일 경우)
⇒ 1. LED 점등
⇒ 2. 촬영에 적절한 위치에 있다는
정보를 서버에게 전달.

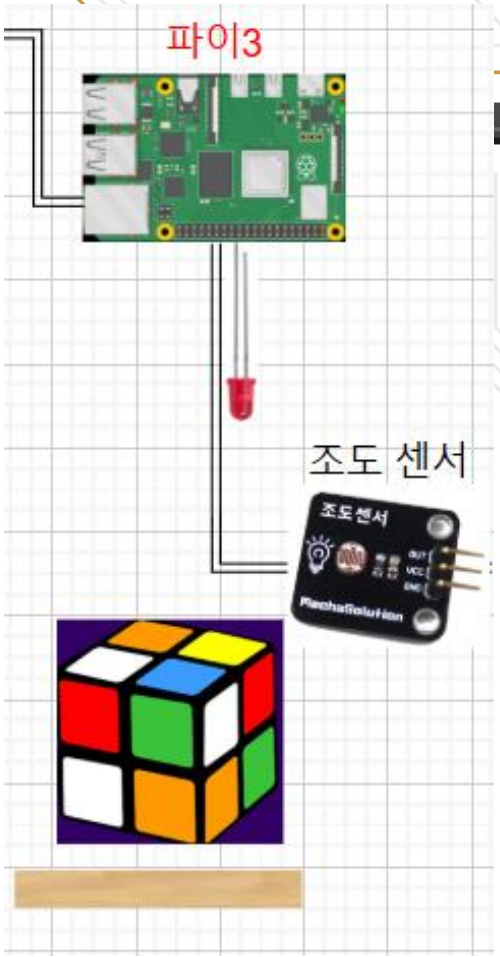


#3번 파이

조도 센서로, 큐브 사진을
제대로 찍을 수 있는 밝기인지 센싱

주변이 어두울 경우, 색을 제대로
인식하지 못함.

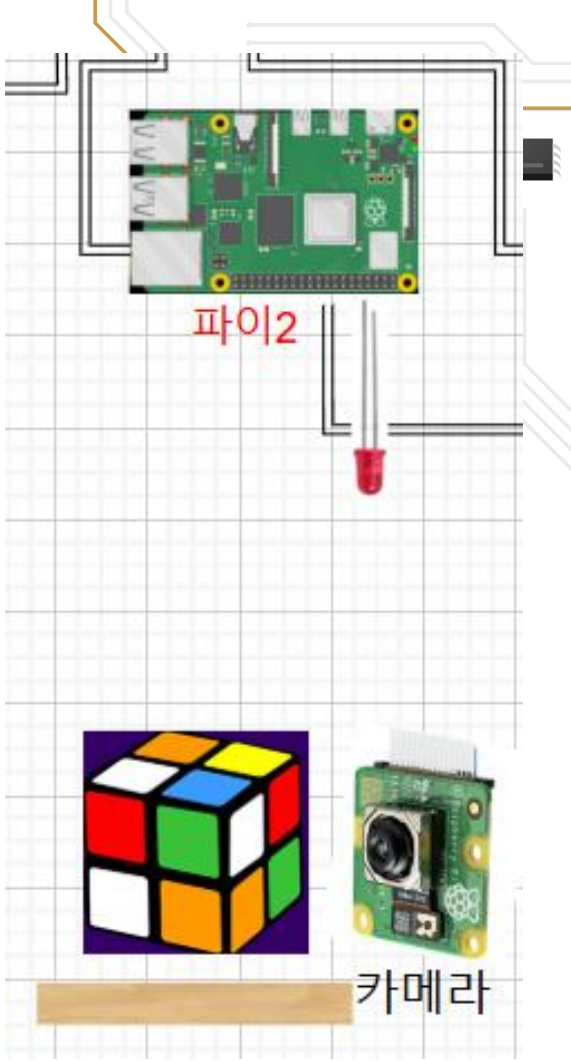
센싱 결과 적정 밝기인 경우)
⇒ 1. LED 점등
⇒ 2. 촬영에 적절한 밝기라는 정보를
서버에게 전달.



#2번 파이

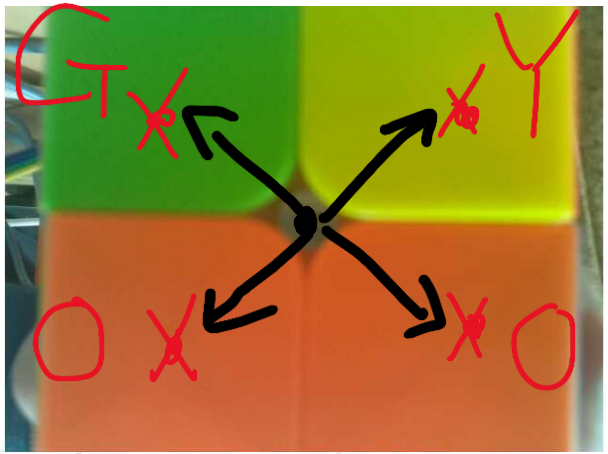
카메라로, 사진을 찍어도 된다고
서버에게 신호를 받기 전까진 대기.

서버에게 촬영 메시지를 받았을 경우)
⇒ 1. 순차적으로 한 면씩 6면을 촬영
⇒ 2. 사진을 찍을 때마다 '찍었음'을
알리기 위해 1초씩 LED 점등

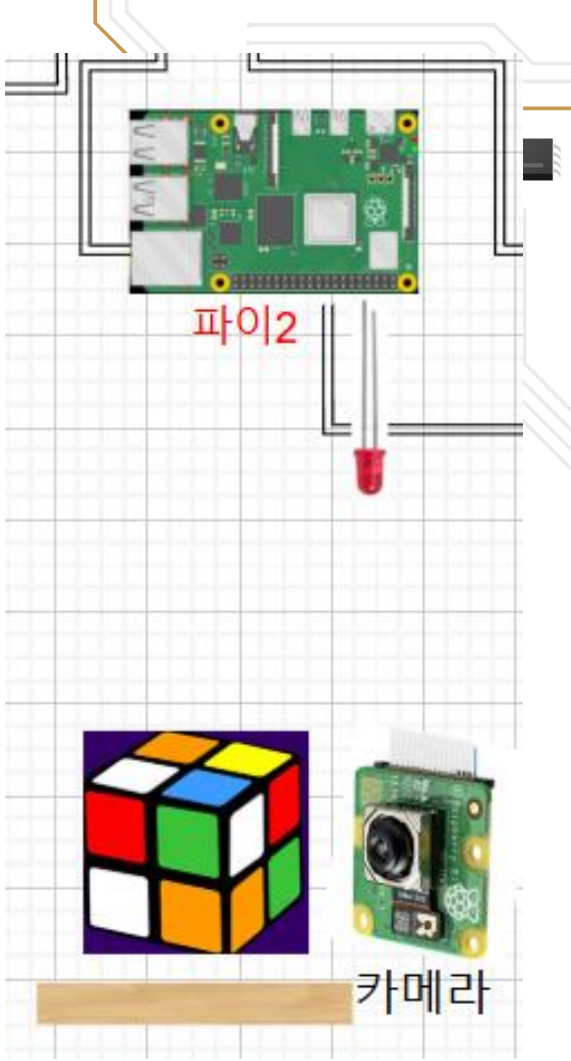


#2번 파이

6면을 모두 촬영한 후,
사진의 특정 위치 픽셀 값을 추출해
현 상태 전개도를 서버에게 전송.



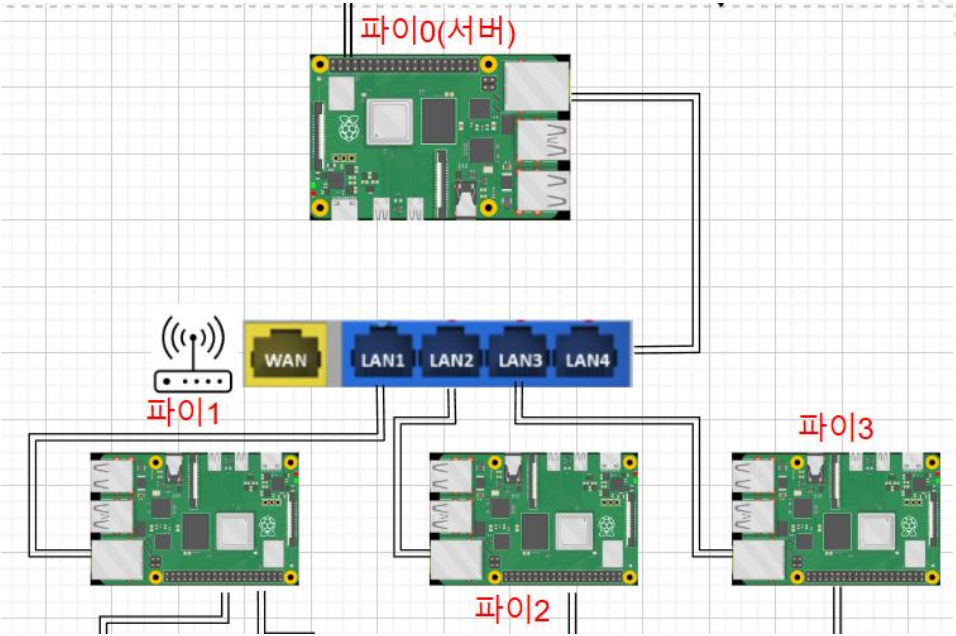
아! 이쪽 면은
[G Y
O O] 구나!!
=> 6면 모두 수행



#0번 파이(서버)

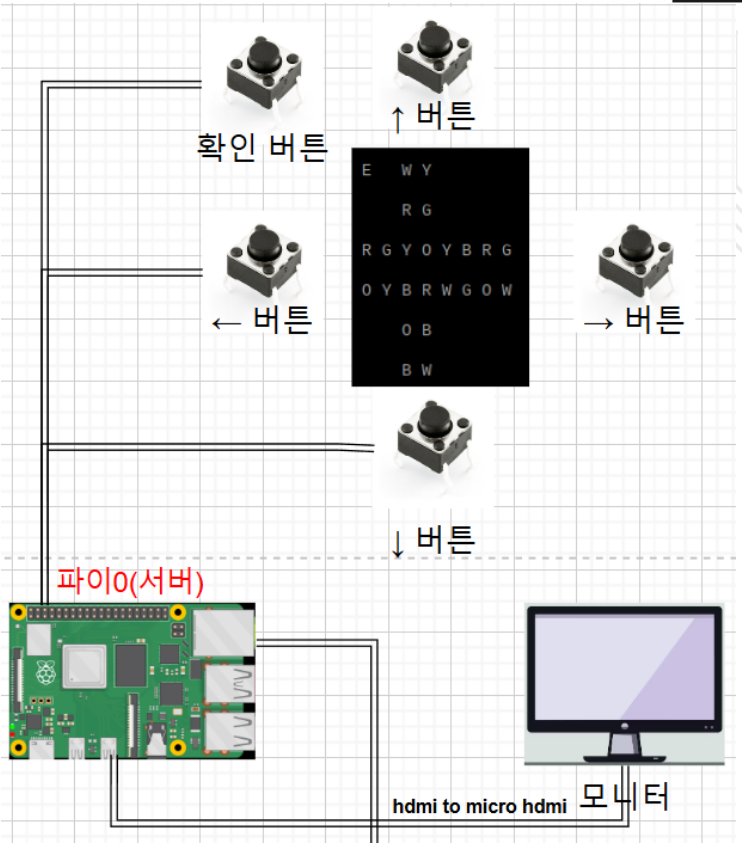
#1번 파이와 #3번 파이의 센싱 값을 전송 받아,

적정 거리 및 적정 밝기일 때 #2번 파이에게 촬영 명령을 내림.



#0번 파이(서버)

추출한 색상의 값이 올바르지 않아
수정이 필요하거나,
환경이 좋지 않아 촬영이 불가능한 경우
=>버튼 조작으로 직접 전개도를
수정/입력함.

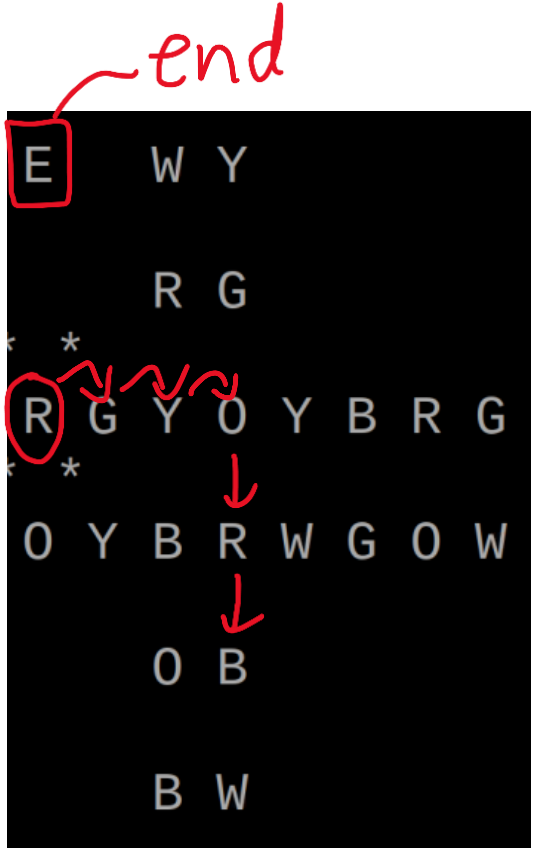


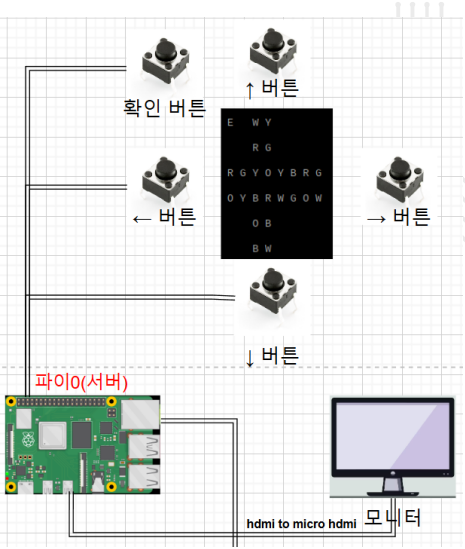
#0번 파이(서버)

4개의 버튼을 사용해
변경할 색으로 이동한 후,
확인 버튼을 눌러 색을 변경.

전개도 수정을 마쳤으면 End
위치로 이동해 확인 버튼을
누르면, 2번 분산 처리 단계로
넘어감.

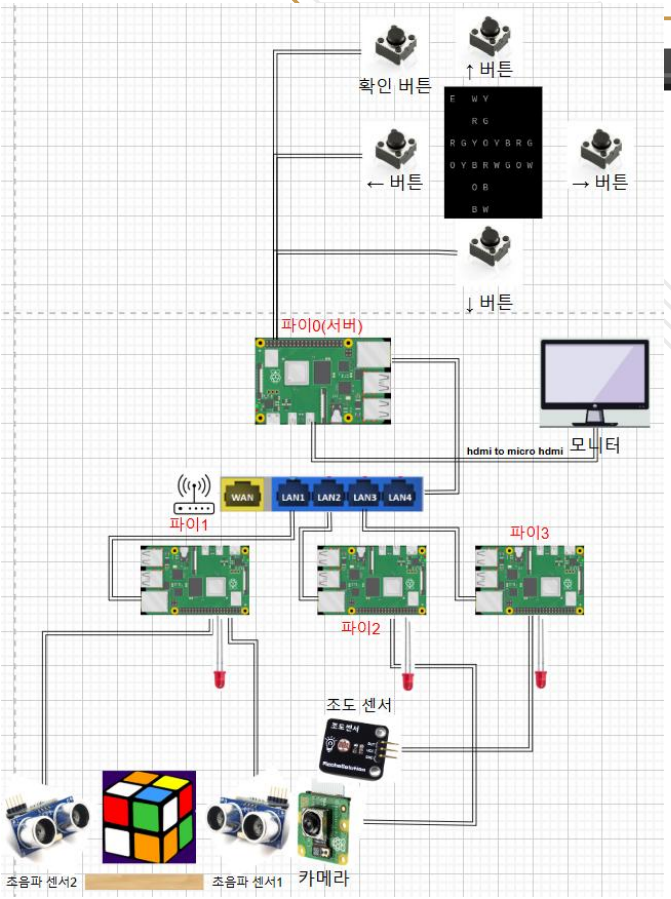
모니터를 통해 전개도 출력





1단계: 입력

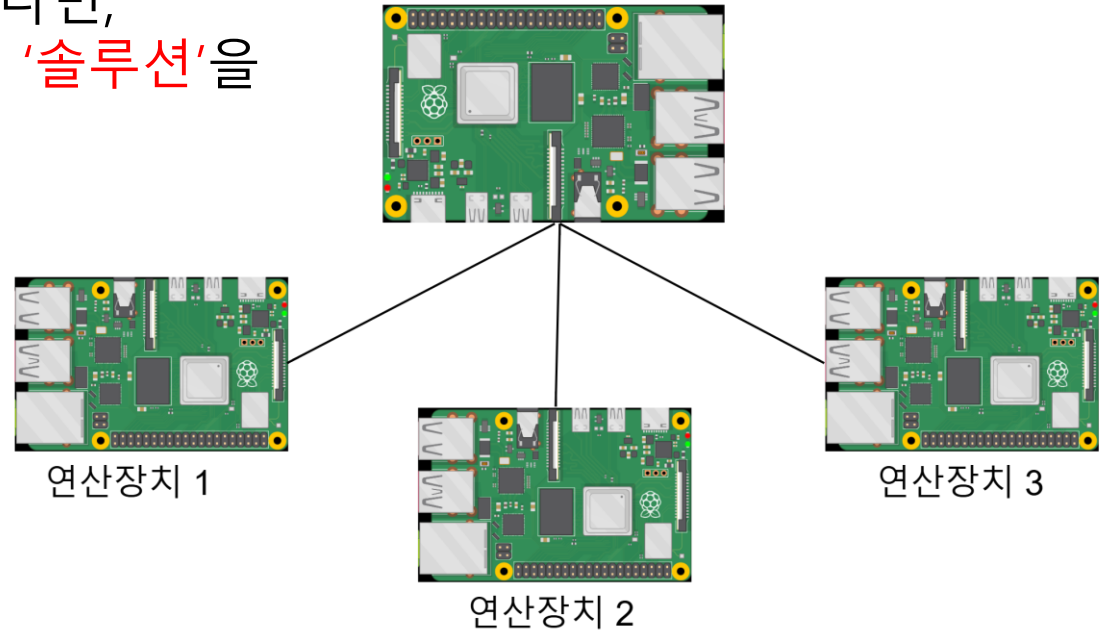
1번 입력 단계의 전체
하드웨어 구조는 우측과 같음.



2단계: 분산 처리

분산 처리의 하드웨어적 연결

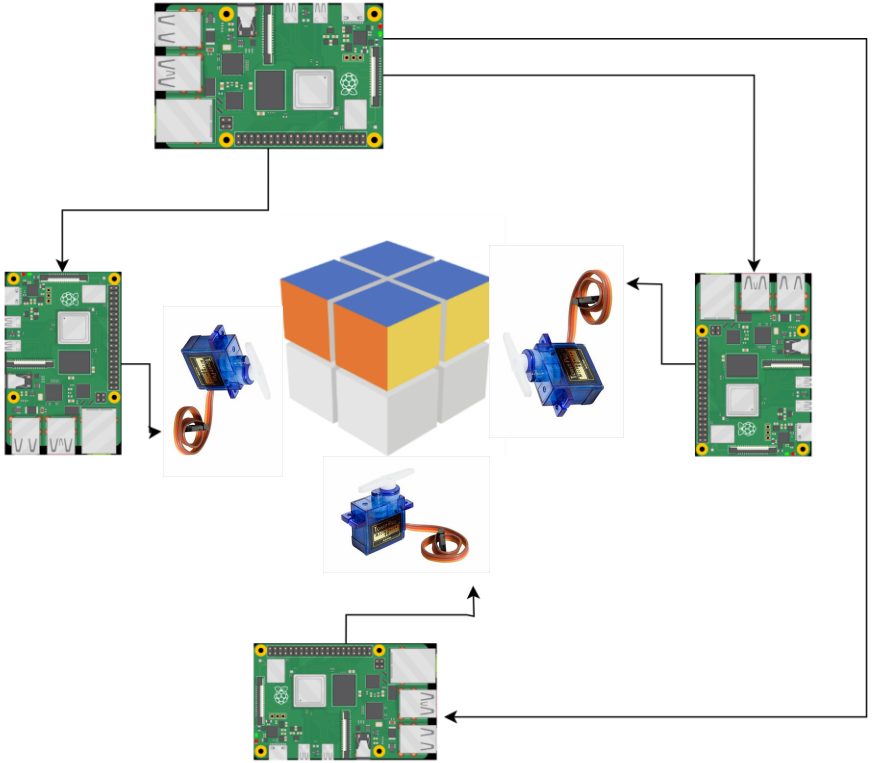
2단계를 수행하고 나면,
XYXXZXYY 와 같은 '솔루션'을
서버가 얻음.

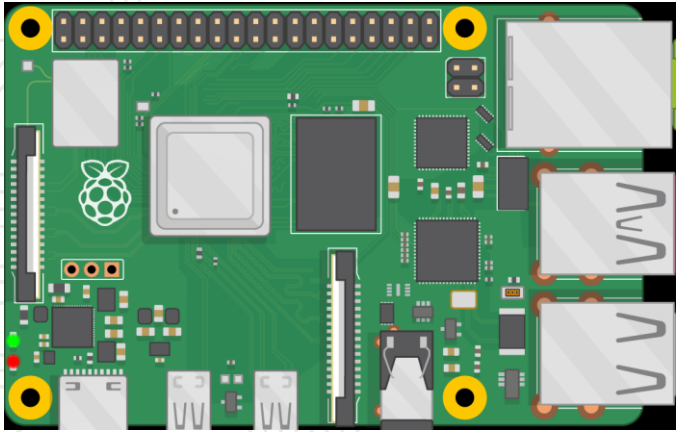


3단계: 출력

서버가 구해준 솔루션에 따라,
서보 모터가 자동으로 큐브를 맞춤.

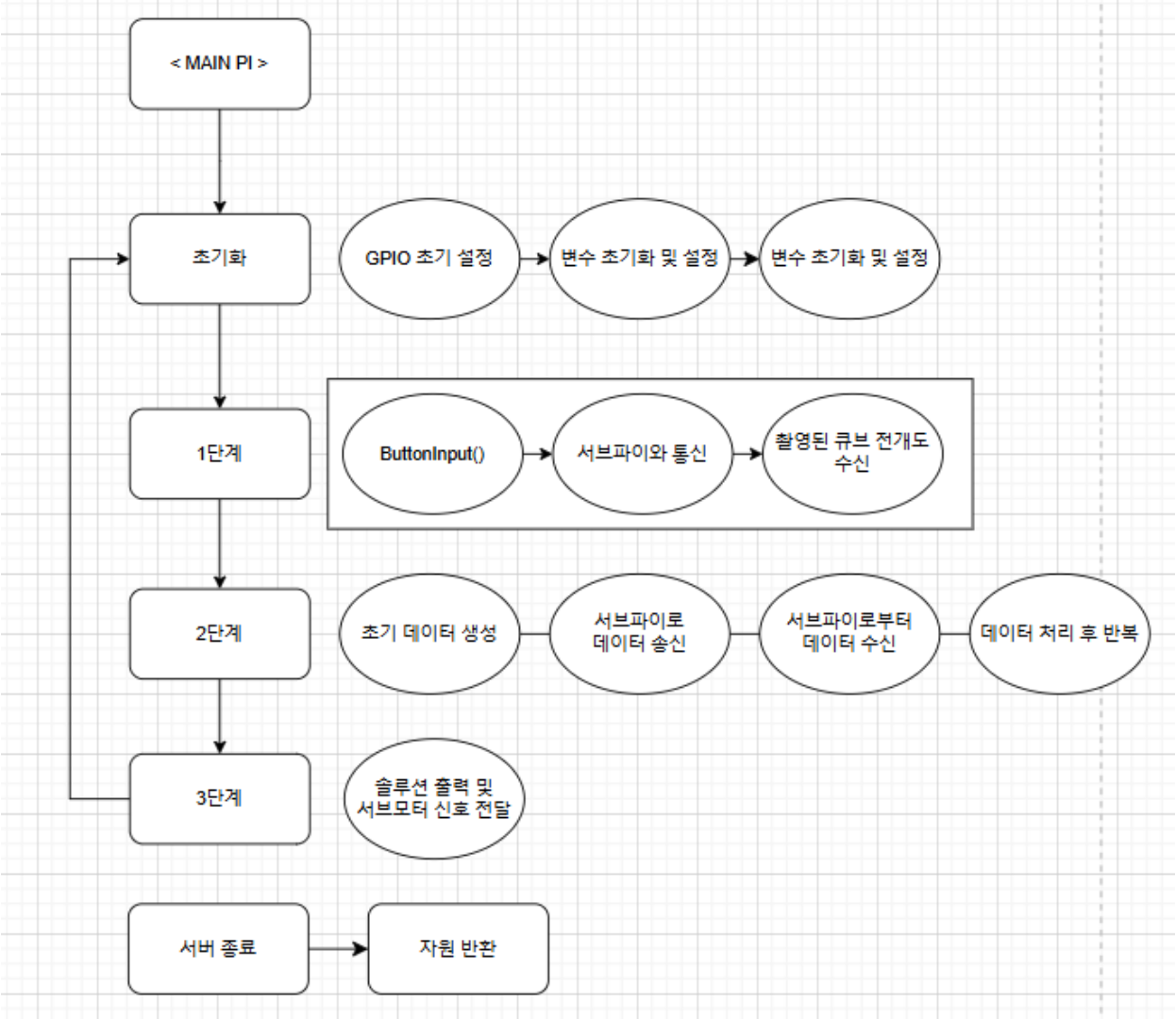
예시) XYXXZXYY

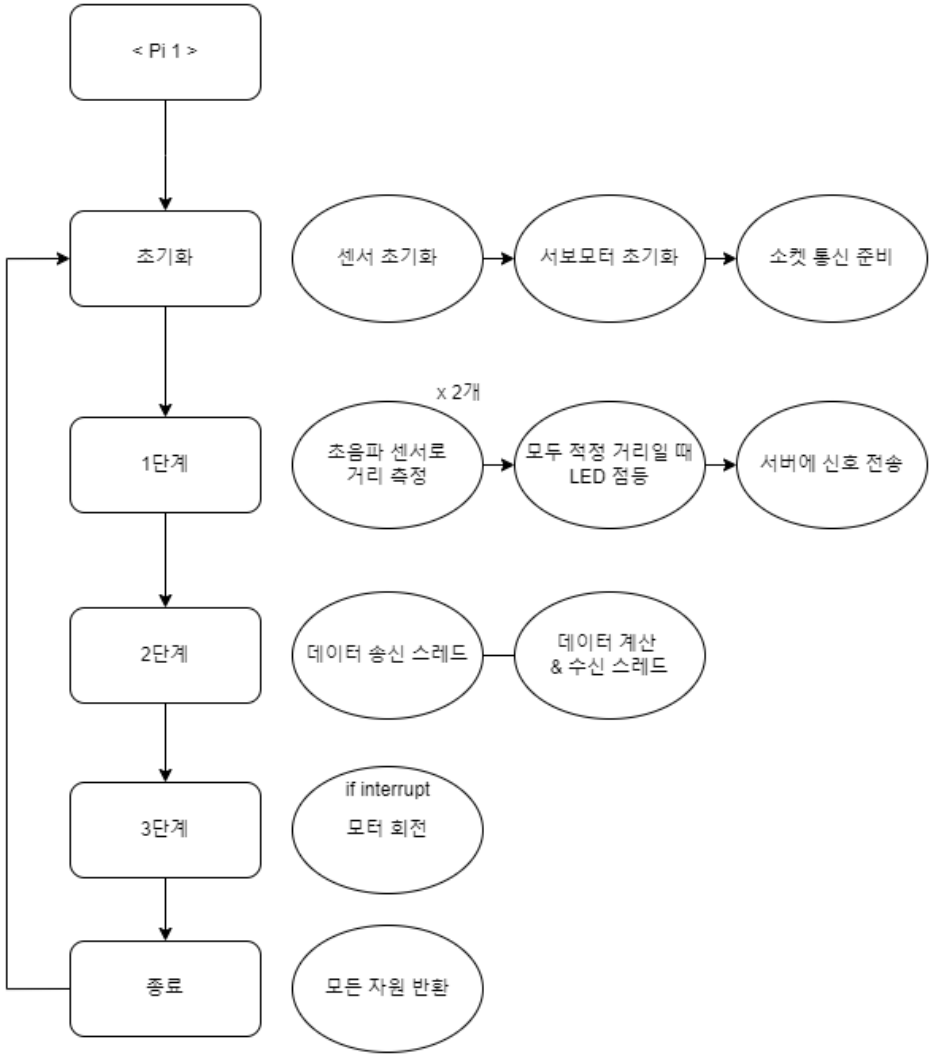


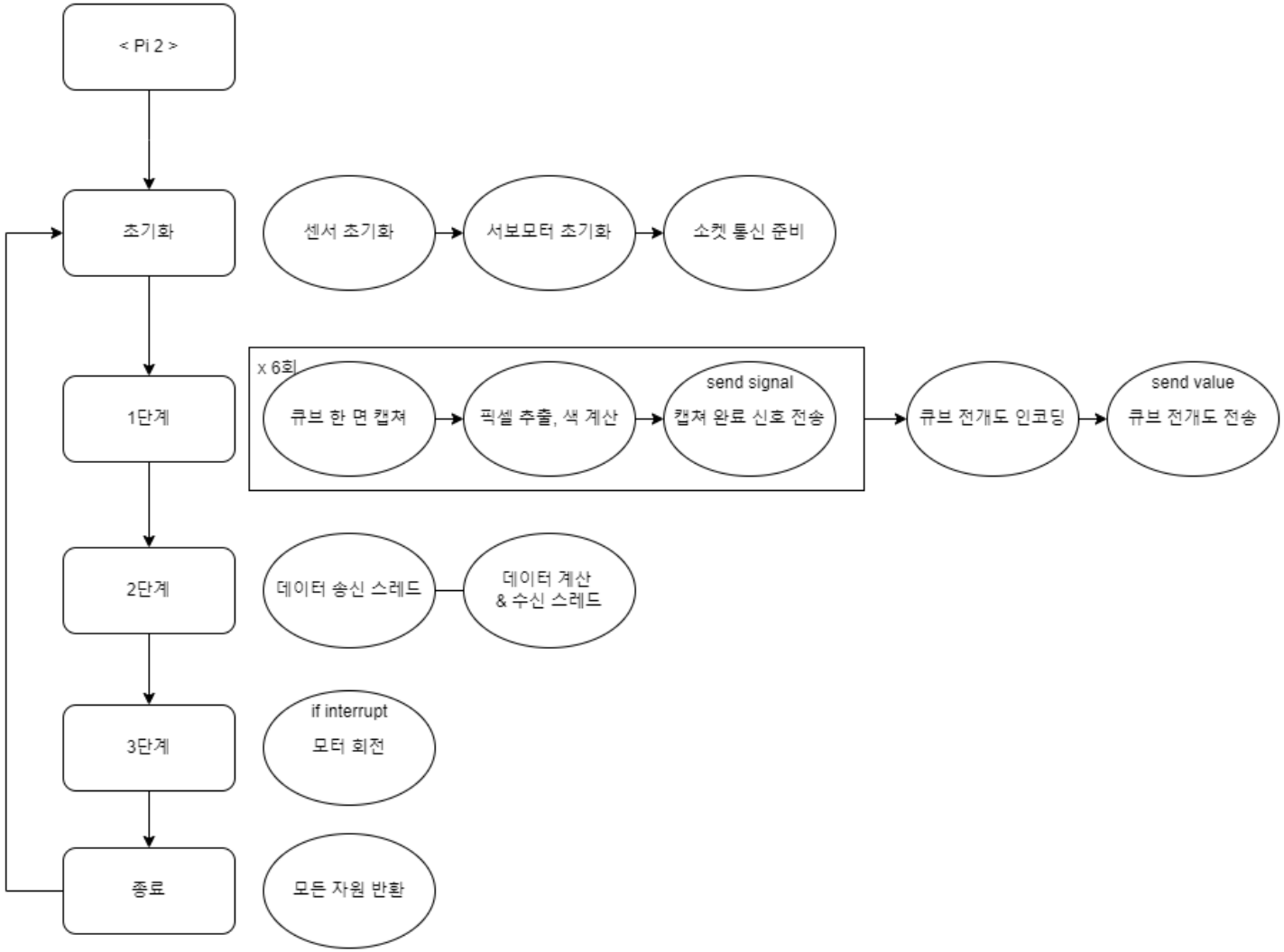


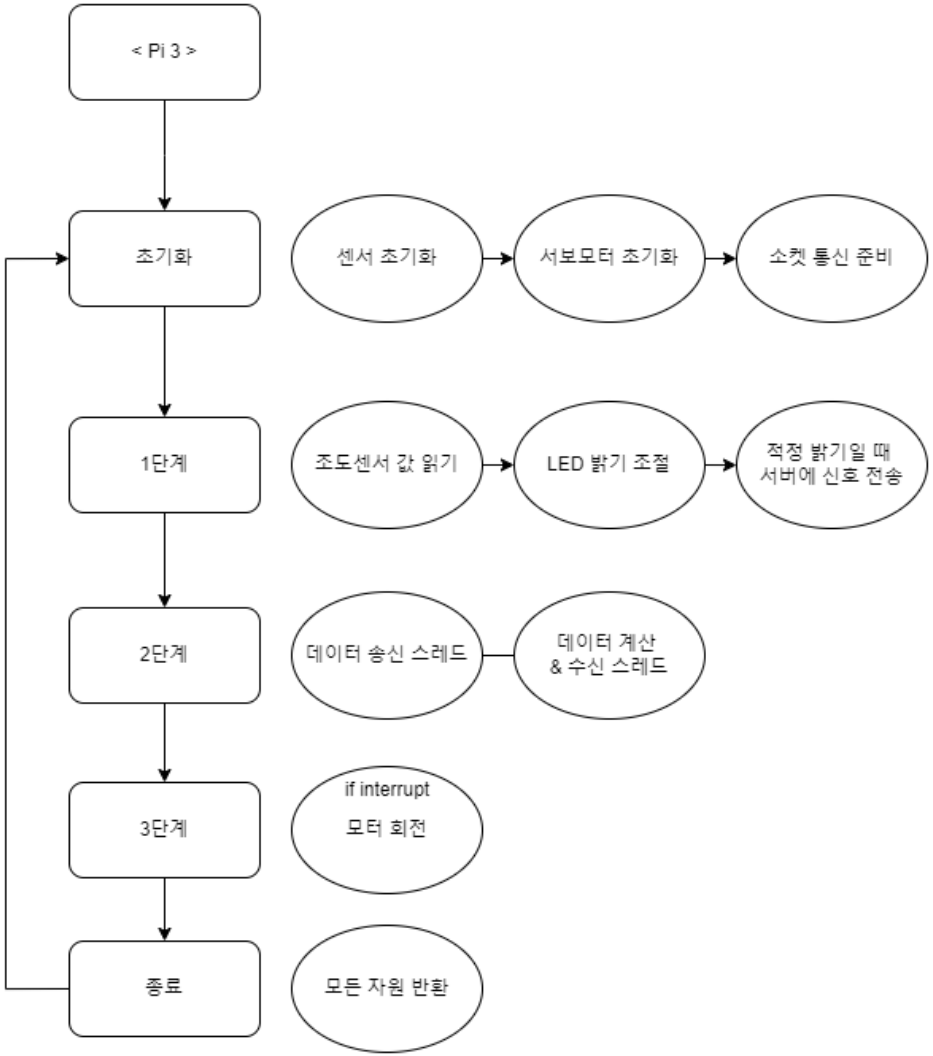
3

소프트웨어 구조 및 설계


< MAIN PI >
초기화
GPIO 초기 설정
변수 초기화 및 설정
변수 초기화 및 설정
1단계
ButtonInput()
서브파이와 통신
촬영된 큐브 전개도 수신
2단계
초기 데이터 생성
서브파이로 데이터 송신
서브파이로부터 데이터 수신
데이터 처리 후 반복
3단계
솔루션 출력 및 서브모터 신호 전달
서버 종료
자원 반환


< Pi 1 >
초기화
센서 초기화
서보모터 초기화
소켓 통신 준비
1단계
x 2개
초음파 센서로
거리 측정
모두 적정 거리일 때
LED 점등
서버에 신호 전송
2단계
데이터 송신 스레드
데이터 계산
& 수신 스레드
3단계
if interrupt
모터 회전
종료
모든 자원 반환


< Pi 2 >
초기화
센서 초기화
서보모터 초기화
소켓 통신 준비
1단계
x 6회
큐브 한 면 캡쳐
픽셀 추출, 색 계산
send signal
캡쳐 완료 신호 전송
큐브 전개도 인코딩
send value
큐브 전개도 전송
2단계
데이터 송신 스레드
데이터 계산
& 수신 스레드
3단계
if interrupt
모터 회전
종료
모든 자원 반환


< Pi 3 >
초기화
센서 초기화
서보모터 초기화
소켓 통신 준비
1단계
조도센서 값 읽기
LED 밝기 조절
적정 밝기일 때
서버에 신호 전송
2단계
데이터 송신 스레드
데이터 계산
& 수신 스레드
3단계
if interrupt
모터 회전
종료
모든 자원 반환

2단계: 분산 처리

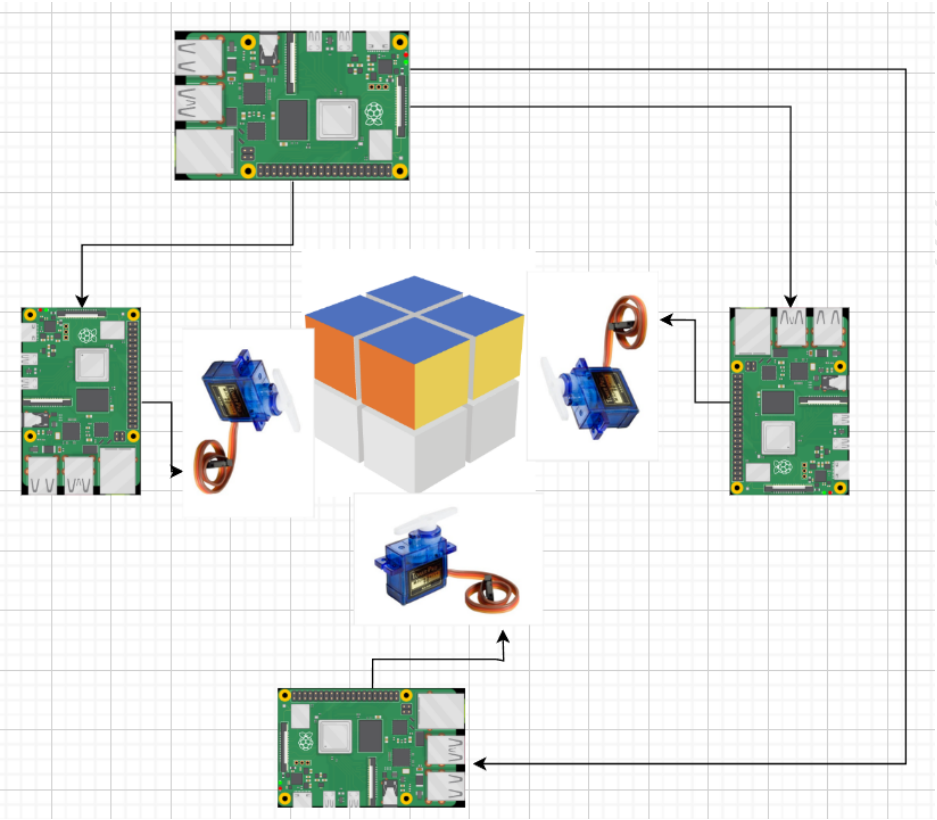
클라이언트는 서버에게 받은 전개도를,

1. 원본
2. X방향 회전
3. Y방향 회전
4. Z방향 회전

총 4개의 전개도로 만들어 다시 서버에게 전송함.

2단계: 분산 처리

1번 입력 단계의 전체
하드웨어 구조는 우측과 같음.

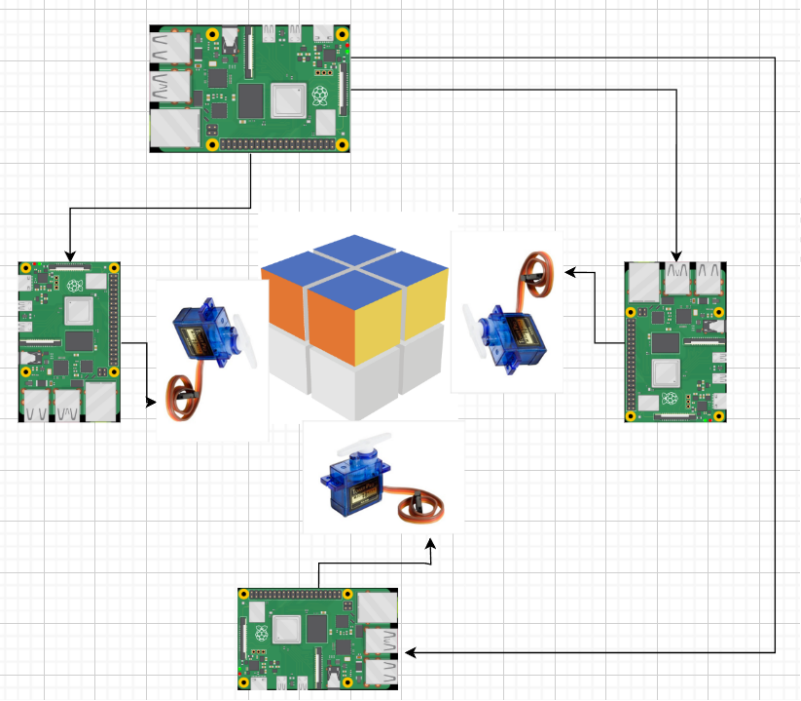


3단계: 출력

PWM을 사용)

period = 10000000일 때,
duty_cycle = 400000이면 => 0도
duty_cycle = 1450000이면 => 90도
duty cycle = 2500000이면 =>180도
에 근사한 위치로 서보 모터가
돌아간다는 것을 발견.

duty_cycle을 조절해 서보 모터를 돌리는
함수 구현.

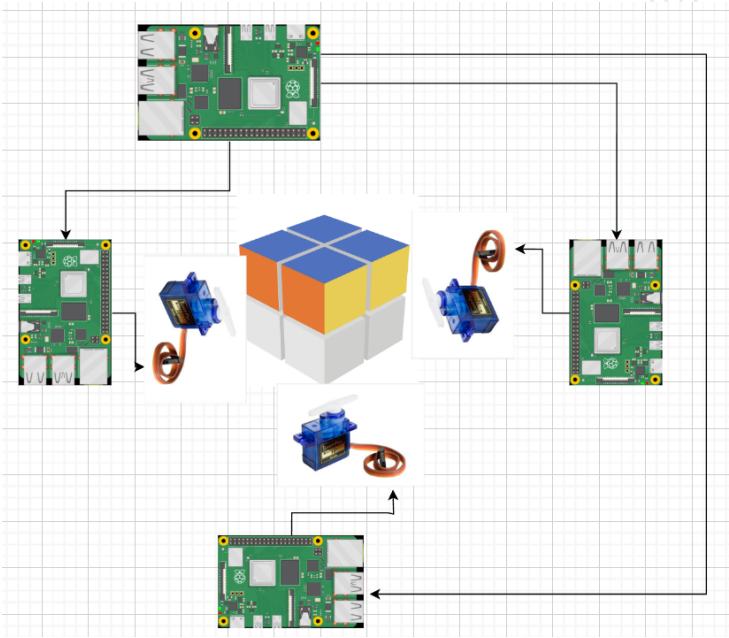


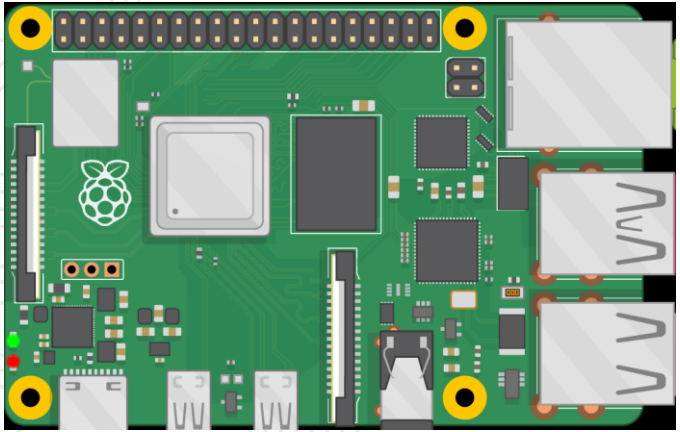
3단계: 출력

서버는 #n번 파이에 달린 서보 모터를 돌리기 위해, n을 클라이언트에게 전송함.

클라이언트는 자기 자신의 flag에 해당하는 메시지를 받을 시 서보모터 회전.

ex. 3을 클라이언트에게 전송 -> #3번 파이를 제외하곤 해당 메시지를 무시, #1번 파이만 해당 메시지를 받아들여 서보모터 회전.





4

기존 계획 대비 변경점

제안 발표에서 예상했던 발생 가능 문제.

서보 모터를 90도 단위로
정확히 돌리지 못한다면??
=> 90도 단위로 돌리는 데 성공.

제안 발표에서 예상했던 발생 가능 문제.

큐브를 회전시키기 위한 모터를 물리적으로 어떻게 배치할 것인가?
=> 필요 이상의 기계공학적 지식과 3D 프린터가 필요.

제안 발표에서 예상했던 발생 가능 문제.

큐브에 서보 모터 직접 연결 x.

"특정 면을 담당하는 서보 모터가 90도 회전했다가 원상태로 돌아왔다" = "서보 모터가 그 면을 회전시켰다"**라고 가정.**

조이스틱 -> 버튼 4개로 대체

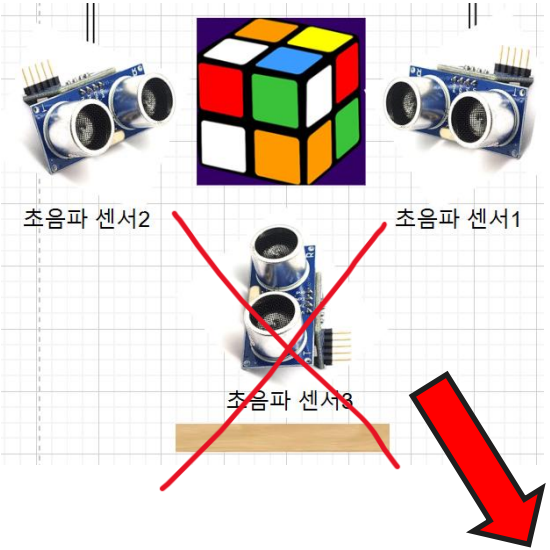
부품 수급 문제로 계획했던
조이스틱 대신, 버튼 4개를 각각
방향키로 구현하여 사용.

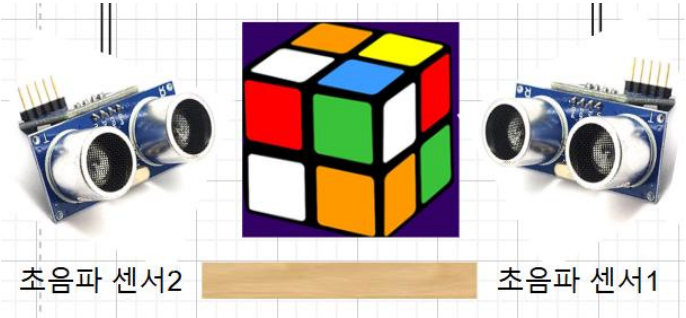
초음파 센서 3개 사용 -> 2개 사용

2개의 수평 초음파 센서만으로도
높이 정보 유추 가능

따라서, 기존에 높이를 센싱하고자
했던 초음파 센서 비사용 결정.

simple structure & 비용 절감



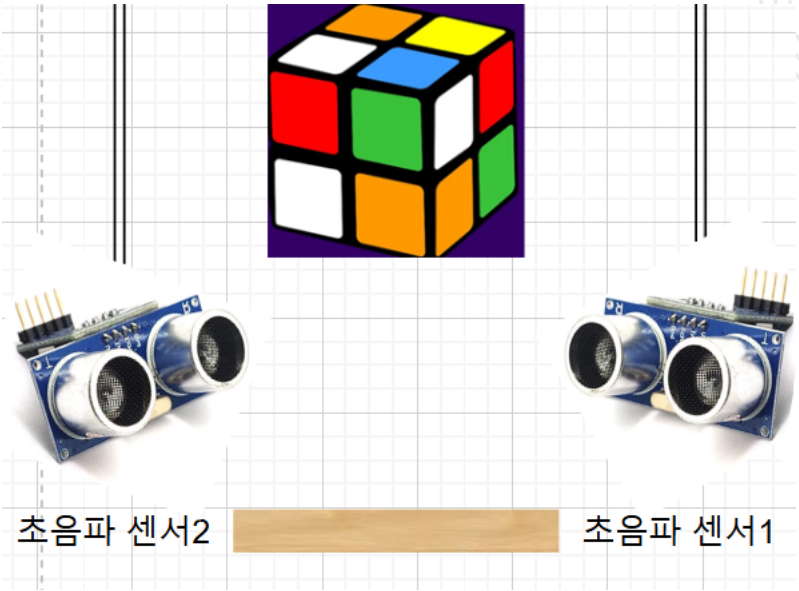


초음파 센서 3개 사용 -> 2개 사용

만약, 우측처럼 큐브가 적절하지 않은 높이에 있다면

초음파 센서가 센싱한 거리 값이 매우 커짐.

⇒ 높이 정보 유추 가능.

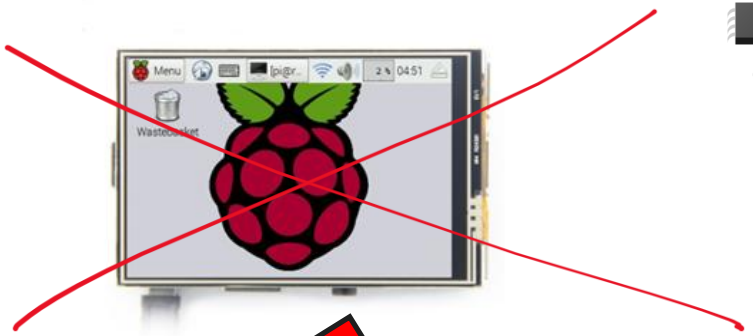


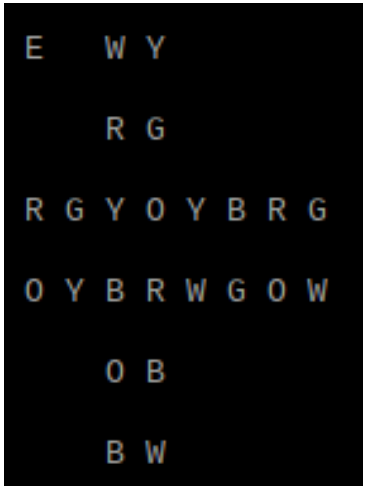
터치스크린 사용 대신 모니터에 연결

전개도 모습을 온전히 보여주기 위해,

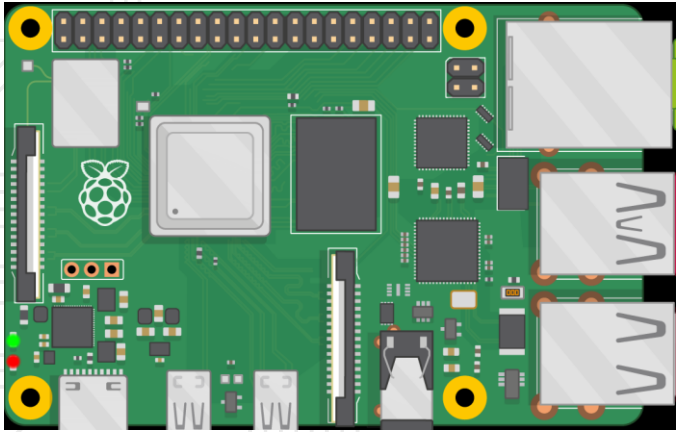
hdmi로 스크린에 연결 후,
터미널 창을 사용자에게 보여줌.

부품 수급 문제로 계획했던
터치스크린 대신 모니터에 연결.





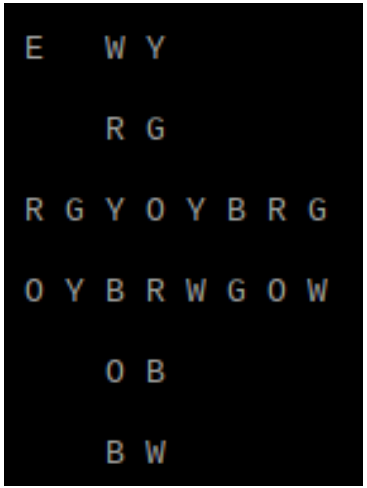
터미널에 출력된 전개도



5

트러블 슈팅 및 시스템 개선 과정

```
static char cube[13][17]={
    {7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7},
    {7,8,7,7,7,1,7,1,7,7,7,7,7,7,7,7,7},
    {7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7},
    {7,7,7,7,7,1,7,1,7,7,7,7,7,7,7,7,7},
    {7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7},
    {7,3,7,5,7,4,7,4,7,5,7,0,7,3,7,4,7},
    {7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7},
    {7,4,7,0,7,3,7,2,7,0,7,1,7,5,7,2,7},
    {7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7},
    {7,7,7,7,7,1,7,5,7,7,7,7,7,7,7,7,7},
    {7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7},
    {7,7,7,7,7,3,7,0,7,7,7,7,7,7,7,7,7},
    {7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7}
};
```



서버와 클라이언트는
Int형 배열 대신,
long long 타입 값을
주고받음.

```
long long l0=4271069981394162510
```

카메라 촬영 & 색상 추출 방식

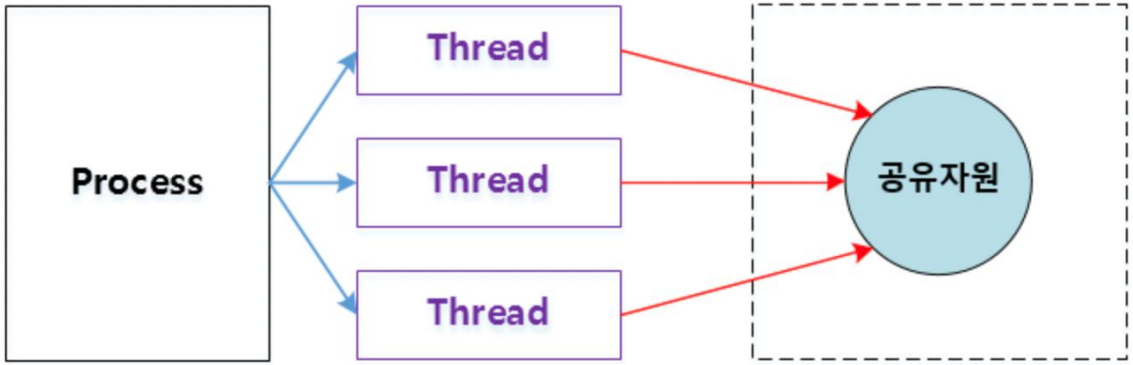
AI 또는 사진 촬영 외부 라이브러리
(Picamera, OpenCV 등)를 사용할 경우 불필요한 자원 낭비 우려

리눅스 기본 제공 V4L2 라는 라이브러리를 사용, C언어로
장치 드라이버를 파일 I/O 함수 직접 조작해 카메라로 촬영함.

OS – process management

메인 파이) 데이터 수신에 스레드 사용 시 임계 구역 문제 발생

뮤텍스 또는 세마포어 사용 시 해결 가능하나, 오버헤드 감당 불가능.



OS – process management

C언어에서의 스레드는 일정 시간을 실행되고 있는 스레드들에게 배분함.

원래는 한 함수가 1초 동안 동작 가능한 것을,
2개의 스레드를 사용하면 한 함수가 0.5초 만큼만 동작 가능

=> 스레드를 사용할 경우, 오히려 느려질 가능성이 있음.

OS – process management

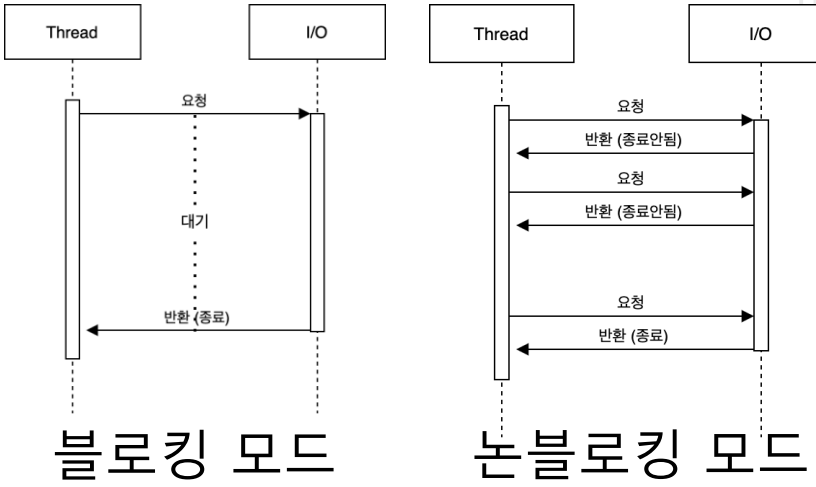
스레드가 필요했던 이유)

Read는 기본적으로 블로킹 모드로 작동

⇒ 대기하는 시간동안 다른 스레드에서 다른 작업을 하여 효율을 높이겠다.

그러나, read를 논 블로킹 모드로 사용 => read가 대기하지 않고 연속적으로 작동하게 됨

⇒ 결국 스레드를 사용할 이유가 없어짐.

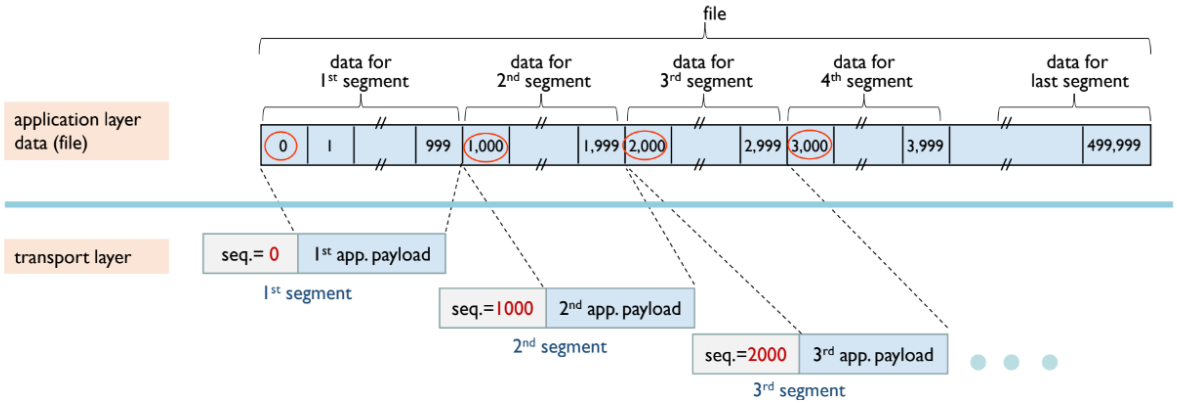


블로킹 모드 논블로킹 모드

Socket programming

배열의 개수가 13개(416byte)일 경우 : 통신 데이터가 분할되지 않고, 한 차례에 전부 송수신 가능하였음.

그러나, 배열의 개수가 14개(448byte)일 경우부터는 데이터가 분할되어 도착함.

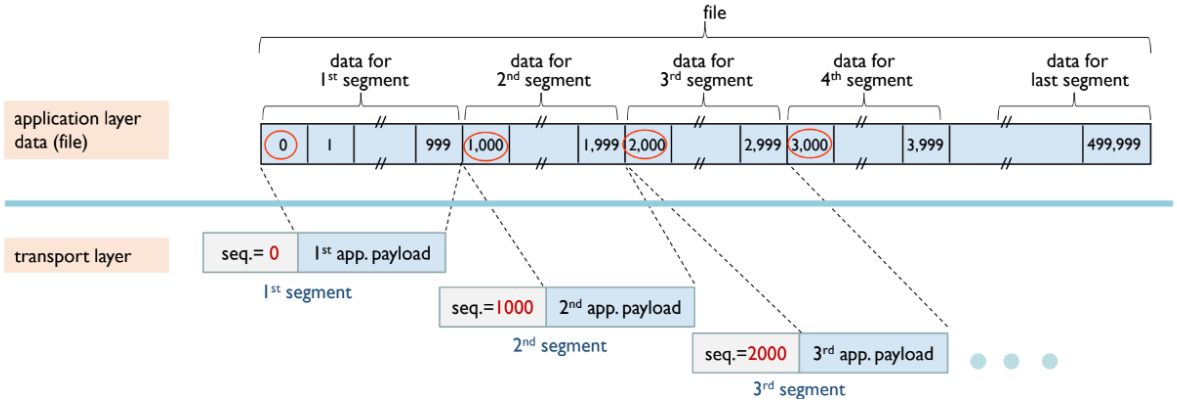


Computer Networking: A Top-Down Approach

Socket programming

배열의 크기를 크게 하여, 데이터를 분할하여 여러 번 보내는 것보다 한 번에 보낼 수 있는 만큼의 데이터양만 보내어, 통신 딜레이를 최소화하고자 하였다.

=> 서브 파이에서는 10개의 데이터까지 가질 수 있는 묶음을 만들어 그 만큼씩 데이터를 송수신함.



Computer Networking: A Top-Down Approach

분산 처리 - 연산 속도 향상

1. 파이 로컬 프로그램 연산 : 약 16초

2. 파이 1개 단일 데이터 : 54초

3. 파이 1개 데이터 10개 : 23초

4. 파이 2개 데이터 10개 : 19초

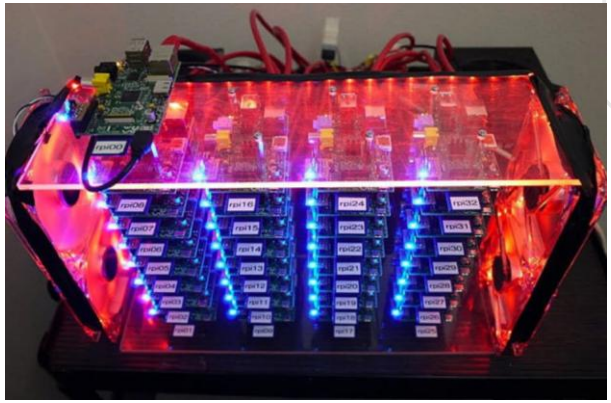
5. 파이 3개 데이터 10개 : 16초 -> 2번과 비교하여 3배 연산 속도 향상

클라이언트 노드 개수 증가

분산 처리를 수행하는 클라이언트 노드 개수가 증가 시

⇒ 분산 처리의 효율이 높아짐.

노드 3개 -> 수십개 -> 수백개 -> .....

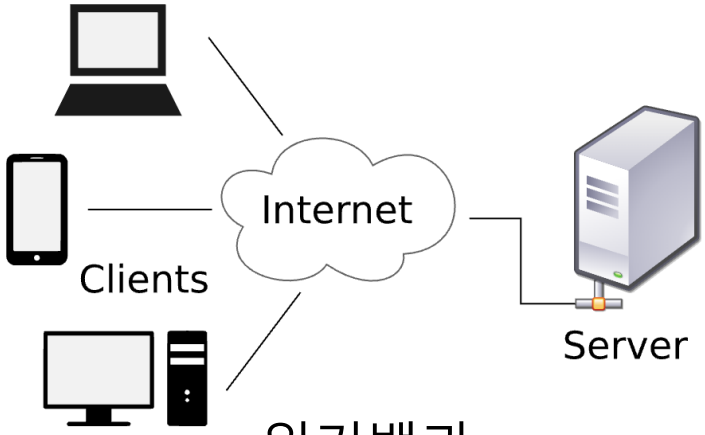


32노드 분산 처리 시스템
By Joshua Kiepert, Boise State University

Robust한 시스템

서버 : 항상 켜져있는 백엔드와 같은 역할

클라이언트 : 통신 접속과 종료가 자유로운 CPU



위키백과

Robust한 시스템

클라이언트 파이는 현재 실행 중인 통신에 대해,
어느 단계에서나 출입이 자유로움.

접속 이후에는 현재 서버가 진행 중인 단계에 참여 가능함.

클라이언트는, 에러가 발생할 경우 Reboot한 후, 자동으로 진행중인 단계에 참여함.

Robust한 시스템

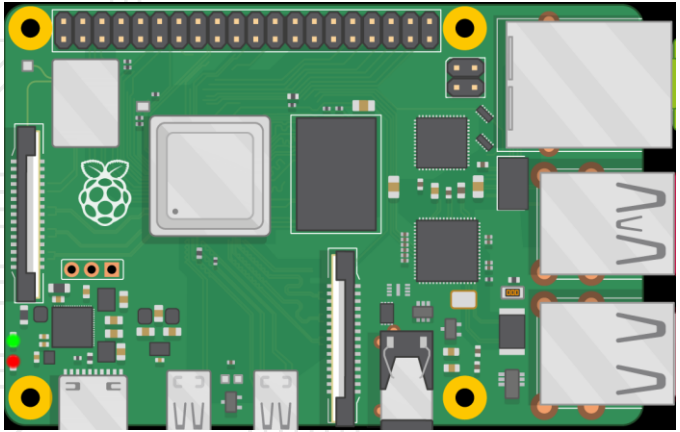
서버는 클라이언트와의 통신이 불가능한 상황에 처했을 때,

1단계 : 버튼을 통해 전개도를 수정 가능

.

2단계 : 하나의 파이라도 아직 통신에 참여 중일 경우, 단일 통신으로 진행. 만약 접속중인 파이가 하나도 없을 경우 대기함.

3단계 : 하나의 파이라도 오류가 있을 경우 화면에 출력된 솔루션을 보고 큐브 솔빙 가능.



6

시스템 확장 가능성

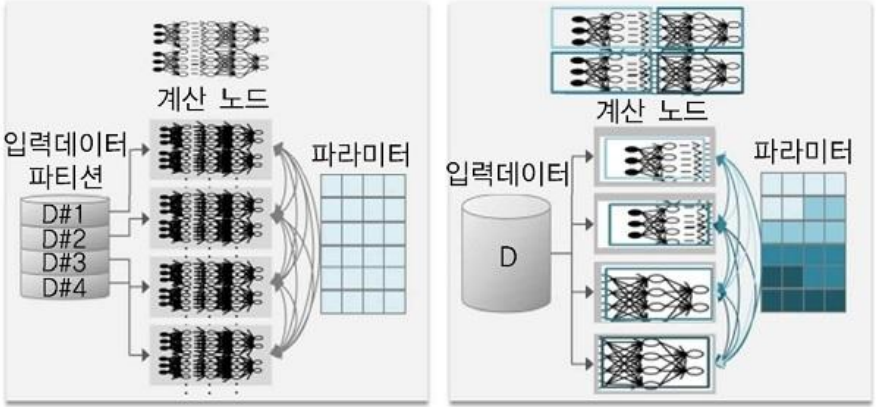
타 기술에 적용 가능

분산처리의 기초가 되는 시스템을 설계 및 구현하였음.

시스템 구조는 유지한 채, '큐브 솔빙'만 다른 연산으로 치환.

딥러닝 등 많은 양의 연산이 필요한 기술들에 적용 가능.

*특히 **연산 시간 > 통신 오버헤드**인 작업들에 적합함.



(a) 데이터 병렬처리 (b) 모델 병렬처리

2016, 딥러닝 분산처리 기술동향 31권 3호, 안신영 외

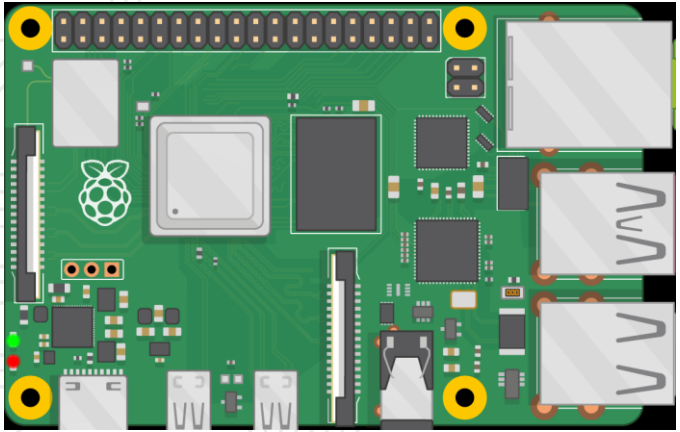
타 기술에 적용 가능

최신 IoT 기기들은 AI를 사용하기 위해
많은 양의 연산을 수행함.

IoT로 둘러싸인 공간을 가정해보면,

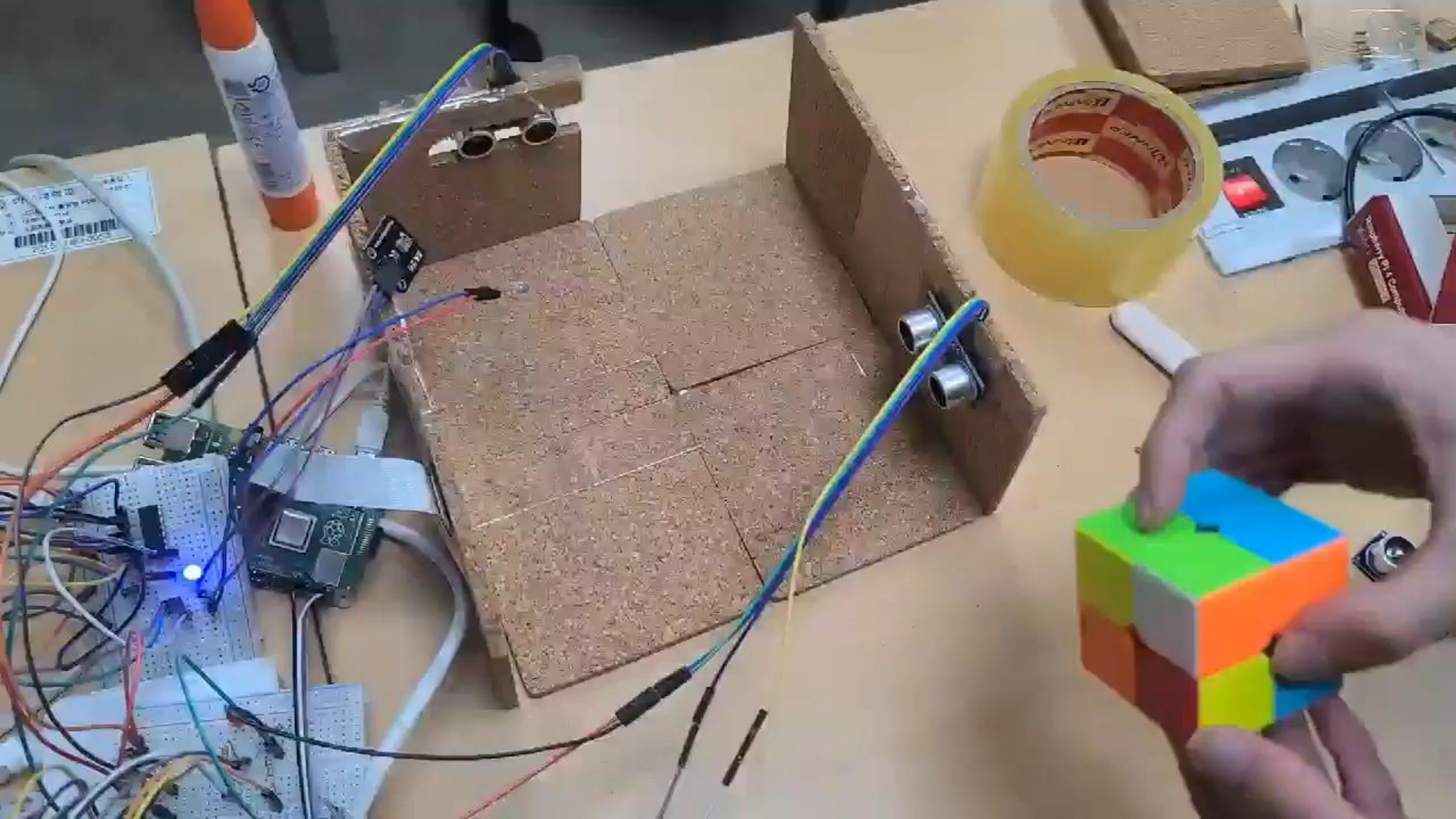
서버 기준에서 현재 대기 상태인
가구들의 CPU를 활용하여 분산 처리
=> 자원 및 시간 절약 가능

KT 기가지니 홈페이지



7

시연 영상



감사합니다.